

Task Management System - Full-Stack Application

A complete task management solution with RESTful API backend and Blazor Server frontend

This project demonstrates a modern full-stack .NET 8 application consisting of: - **Backend API**: RESTful web services with JWT authentication (Part 1) - **Frontend Web App**: Blazor Server interactive UI (Part 2) - **Comprehensive Testing**: 74 unit tests with ~95% controller coverage

Table of Contents

1. What is This Project?
2. Technology Stack Explained
3. System Architecture
4. Part 1: Backend API (Summary)
5. Part 2: Blazor Frontend Application
6. Project Setup & Running
7. Testing Documentation
8. Security Features
9. Project Structure

What is This Project?

A **task management application** where users can: - Register and login securely (JWT tokens, like OAuth) - Create projects to organize work - Add tasks to projects with status tracking (To Do → In Progress → Done) - Search and filter tasks by project or status - Only see and manage their own data (authorization enforced)

Think of it as: A simplified Trello or Asana, built to demonstrate enterprise-grade patterns.

Technology Stack Explained

Backend (.NET 8 Web API)

- Language: C# (like Java/TypeScript, but for .NET)
- Framework: ASP.NET Core (web framework, like Express.js or Spring Boot)
- Database: SQLite (file-based SQL database, like a mini-PostgreSQL)
- ORM: Entity Framework Core (database abstraction, like Hibernate or TypeORM)
- Authentication: JWT tokens (JSON Web Tokens, like OAuth without external provider)
- Testing: xUnit (unit testing framework, like Jest or JUnit)

Frontend (Blazor Server)

- Framework: Blazor Server (component-based UI, like React but server-side)
- Language: C# + Razor (HTML templating, similar to JSX)
- State Management: Built-in component state (like React hooks)
- Styling: Bootstrap 5 (CSS framework)
- HTTP Client: HttpClient (like Axios or Fetch API)

What is Blazor?

Blazor is Microsoft's framework for building interactive web UIs using C# instead of JavaScript.

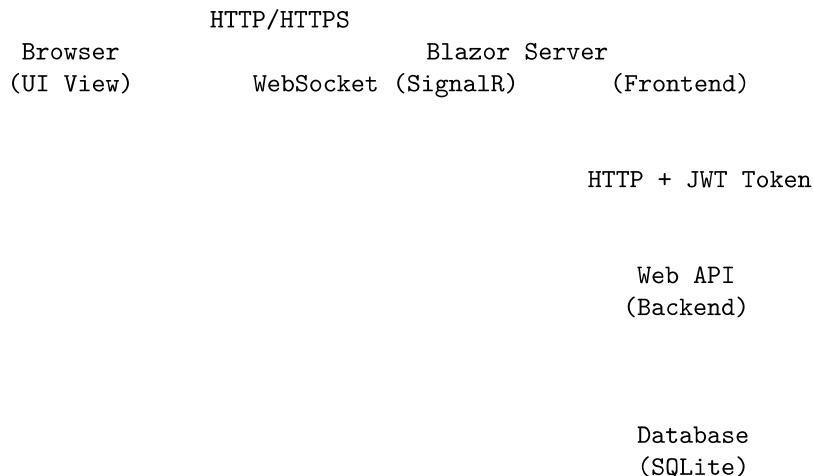
Blazor Server (what we use here) runs your UI logic on the server and updates the browser in real-time over a WebSocket connection (SignalR). Think of it like **React**, but the components run on the server, not in the browser. User interactions trigger server-side C# code, which updates the UI.

Why use it? - Write frontend and backend in the same language (C#) - Type safety end-to-end (no "undefined is not a function") - Less JavaScript complexity (no npm, webpack, etc.) - Good for enterprise line-of-business apps

Alternative modes: Blazor WebAssembly runs C# in the browser (like a SPA), Blazor Auto picks the best mode dynamically.

System Architecture

High-Level Overview



Data Flow Example: Creating a Project

1. User fills form in browser
2. Browser sends interaction to Blazor Server (SignalR)
3. Blazor calls `ProjectApiService.CreateProjectAsync()`
4. `ApiClient` adds JWT token to HTTP request
5. API receives `POST /api/project` with Bearer token
6. API validates token, extracts user ID
7. `ProjectController` calls `ProjectService`
8. `ProjectService` saves to database via EF Core
9. Response flows back: API → Blazor → Browser UI updates

Authentication Flow

User

Backend

1. `POST /api/auth/login`
`{ username, password }`
2. Verify password (BCrypt)
Generate JWT token
3. Return `{ token, userId, username }`
4. Store token in `localStorage`
5. All future requests include token
`Authorization: Bearer <token>`
6. API validates token, extracts user ID
Enforces ownership (only see your data)

Part 1: Backend API (Summary)

Developed by team of 3 students with role separation: - **Student A (DB Lead)**: Models, migrations, seed data - **Student B (Services)**: Controllers, services, DTOs - **Student C (Security)**: Authentication, JWT, authorization

Core Components

1. Data Models

- **User:** Authentication, owns projects
- **Project:** Container for tasks, owned by user
- **TaskItem:** Individual work items with status (ToDo/InProgress/Done)
- **Comment:** Optional discussion on tasks

2. Service Layer Business logic with **ownership enforcement:** -
ProjectService: CRUD for projects (user sees only their projects) -
TaskService: CRUD + status management + assignment - CommentService:
CRUD for task comments - IUserContext: Abstraction to get current user ID
from JWT token

3. API Endpoints Authentication (public):

POST /api/auth/register - Create account
POST /api/auth/login - Get JWT token

Projects (requires authentication):

GET	/api/project	- List user's projects (with search)
POST	/api/project	- Create project
GET	/api/project/{id}	- Get project details
PUT	/api/project/{id}	- Update project
DELETE	/api/project/{id}	- Delete project (cascades to tasks)

Tasks (requires authentication):

GET	/api/task	- List user's tasks (filter by project/status)
POST	/api/task	- Create task
GET	/api/task/{id}	- Get task details
PUT	/api/task/{id}	- Update task
PATCH	/api/task/{id}/status	- Change status
PATCH	/api/task/{id}/assign	- Assign to user
DELETE	/api/task/{id}	- Delete task

Comments (requires authentication):

GET	/api/comment/task/{taskId}	- Get comments for task
POST	/api/comment	- Add comment
PUT	/api/comment/{id}	- Update comment
DELETE	/api/comment/{id}	- Delete comment

4. Security Features

- **JWT Authentication:** Bearer tokens (like OAuth, self-contained)
- **Password Hashing:** BCrypt with salt (never store plain text)
- **Ownership Checks:** Service layer filters by current user

- **Authorization:** Returns 404 for unauthorized access (not 403, avoids leaking info)

5. Database

- **SQLite:** File-based database (`tasks.db`)
- **Entity Framework Core:** ORM (no raw SQL needed)
- **Migrations:** Version-controlled schema changes
- **Seed Data:** Pre-populated users (alice, bob) for testing

6. Testing (Part 1)

- **74 unit tests** with xUnit
- **~95% controller coverage**
- In-memory SQLite for test isolation
- Tests: CRUD operations, authorization, validation, edge cases

Part 2: Blazor Frontend Application

Objective: Build a web UI that consumes the API and provides user-friendly task management.

What Was Implemented

A complete **Blazor Server** application with:

1. Authentication Module Login & Registration with JWT token management

- **Components:**
 - `LoginComponent.razor`: Login form with validation
 - `RegisterComponent.razor`: User registration form
- **Services:**
 - `AuthenticationService`: Handles login/register API calls
 - Stores JWT token in browser's `localStorage` (persists across page refreshes)
 - Decodes JWT to extract user ID and username
- **Flow:**
 1. User logs in → API returns JWT token
 2. Token saved to `localStorage`
 3. Every API call includes token in `Authorization: Bearer <token>` header
 4. User stays logged in until token expires or logout

Key Implementation: Token retrieved from `localStorage` on **every request** (not stored in memory), solving common persistence issues with Blazor.

2. Project Management Module Full CRUD for projects with search

- **Components:**
 - `ProjectListComponent`: Display all user's projects in table/cards
 - `ProjectDetailComponent`: View single project with task list
 - `CreateProjectComponent`: Form to create new project
 - `EditProjectComponent`: Form to update existing project
- **Features:**
 - Search projects by name (client-side filtering)
 - Task count displayed per project
 - Delete with confirmation dialog
 - Navigation between views
- **Service:** `ProjectApiService` wraps all project API calls

3. Task Management Module Full CRUD for tasks with filtering and status management

- **Components:**
 - `TaskListComponent`: Display tasks with filters (project, status)
 - `TaskDetailComponent`: View task details
 - `CreateTaskComponent`: Form to create task (with project selection)
 - `EditTaskComponent`: Form to update task
- **Features:**
 - Filter by project (dropdown)
 - Filter by status: To Do / In Progress / Done
 - Combined filters (project + status together)
 - Status updates (quick status change buttons)
 - Task assignment (assign to self)
 - Due date selection
- **Service:** `TaskApiService` wraps all task API calls

4. Shared UI Components Reusable components for consistent UX

- `LoadingSpinner`: Shows during API calls
- `ErrorAlert`: Displays error messages (dismissible)
- `SuccessToast`: Shows success notifications
- `ConfirmDialog`: Confirmation prompts for delete operations

5. Layout & Navigation

- `MainLayout.razor`: Main page structure with navbar
- Navigation links: Home → Projects → Tasks
- Logout button (clears token)
- Responsive design with Bootstrap 5

Frontend Architecture Decisions

Why Blazor Server?

- **Single language:** C# for both frontend and backend (no JavaScript needed)
- **Type safety:** Compile-time checks, no runtime “undefined” errors
- **Simpler deployment:** No separate SPA build process
- **Good fit:** Line-of-business apps with trusted network (not public internet at scale)

API Communication Pattern

```
// ApiClient.cs - Base HTTP client
public async Task<T> GetAsync<T>(string endpoint)
{
    // 1. Retrieve JWT token from localStorage
    var token = await _localStorage.GetItemAsync<string>("authToken");

    // 2. Create request with Authorization header
    request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);

    // 3. Send request to API
    var response = await _httpClient.SendAsync(request);

    // 4. Deserialize JSON response
    return await response.Content.ReadFromJsonAsync<T>();
}
```

Why per-request token retrieval?

Blazor Server creates new service instances per scope. Storing token in HttpClient headers doesn't persist. Solution: fetch from localStorage each time (small overhead, reliable).

Prerendering Disabled All interactive pages use:

```
@rendermode @(new InteractiveServerRenderMode(prerender: false))
```

Why? Prerendering runs C# code **before JavaScript is available**. LocalStorage requires JavaScript, so accessing it during prerender causes errors. Disabling prerender fixes this (trade-off: slight delay before page becomes interactive).

Frontend Folder Structure

```
TaskManagementAPI.Blazor/
  Components/
    Auth/
      LoginComponent.razor      - Login form
      RegisterComponent.razor   - Registration form
    Projects/
      ProjectListComponent.razor - List all projects
```

ProjectDetailComponent.razor	- View single project
CreateProjectComponent.razor	- Create new project
EditProjectComponent.razor	- Edit project
Tasks/	
TaskListComponent.razor	- List tasks with filters
TaskDetailComponent.razor	- View single task
CreateTaskComponent.razor	- Create new task
EditTaskComponent.razor	- Edit task
Shared/	
LoadingSpinner.razor	- Loading indicator
ErrorAlert.razor	- Error message display
SuccessToast.razor	- Success notifications
ConfirmDialog.razor	- Confirmation modal
Layouts/	
MainLayout.razor	- Main page layout
Pages/	
HomePage.razor	- Landing page (/)
LoginPage.razor	- Login route (/login)
RegisterPage.razor	- Register route (/register)
Services/	
IApiClient.cs	- HTTP client interface
ApiClient.cs	- Base HTTP client implementation
AuthenticationService.cs	- Auth operations
ProjectApiService.cs	- Project API calls
TaskApiService.cs	- Task API calls
Models/	
AuthModels.cs	- Auth DTOs
ProjectModels.cs	- Project DTOs
TaskModels.cs	- Task DTOs
wwwroot/	
app.css	- Custom styles
App.razor	- Root component
Routes.razor	- Route definitions
Program.cs	- DI configuration

User Workflows

Creating a Task

1. User navigates to /tasks
2. Clicks "+ New Task" button
3. CreateTaskComponent loads with empty form
4. User fills: title, description, due date, selects project
5. Clicks "Save"
6. TaskApiService.CreateTaskAsync() called
7. ApiClient adds JWT token to POST request

8. API validates, saves to database
9. Success toast shows "Task created!"
10. Redirects to task list

Filtering Tasks

1. User on /tasks page
 2. Selects project from dropdown (e.g., "Website Redesign")
 3. TaskListComponent calls TaskApiService.GetTasksByProjectAsync(projectId)
 4. API returns tasks for that project only
 5. Additionally selects status (e.g., "In Progress")
 6. Combined filter: project + status
 7. UI updates to show matching tasks
-

Project Setup & Running

Prerequisites

- .NET 8 SDK installed ([Download here](#))
- Code editor (Visual Studio 2022, VS Code, or Rider)
- Terminal/PowerShell

Step 1: Clone and Restore

```
# Clone repository (or extract zip)
cd d:\StudioProjects\TaskManagementAPI

# Restore NuGet packages
dotnet restore

# Build solution (compiles everything)
dotnet build
```

Step 2: Configure JWT Secret

Option A: User Secrets (recommended for development)

```
# Navigate to API project
cd TaskManagementAPI

# Initialize secrets
dotnet user-secrets init

# Set JWT key (minimum 32 characters)
dotnet user-secrets set "Jwt:Key" "your-secret-key-min-32-characters-long-123456"
```

```
dotnet user-secrets set "Jwt:Issuer" "TaskApi"
dotnet user-secrets set "Jwt:Audience" "TaskApiClients"
```

Option B: appsettings.json (for demo/testing) Edit TaskManagementAPI/appsettings.Development

```
{
  "Jwt": {
    "Key": "your-secret-key-min-32-characters-long-123456",
    "Issuer": "TaskApi",
    "Audience": "TaskApiClients"
  }
}
```

Step 3: Setup Database

```
# Apply migrations (creates tasks.db)
cd TaskManagementAPI
dotnet ef database update

# Database will be created with seed data:
# - User: alice / password: password123
# - User: bob / password: password123
```

Step 4: Run the Application

Combined (API + Blazor in one process):

```
# From solution root
cd TaskManagementAPI
dotnet run
```

Application runs on: <http://localhost:5114> (API + Blazor)

Separate (API and Blazor independently):

```
# Terminal 1: Run API
cd TaskManagementAPI
dotnet run --urls "http://localhost:5114"

# Terminal 2: Run Blazor (if needed separately)
cd TaskManagementAPI.Blazor
dotnet run --urls "http://localhost:5115"
```

Step 5: Access the Application

Blazor Frontend: <http://localhost:5114>

Swagger API Docs: <http://localhost:5114/swagger>

Test Login Credentials: - Username: `alice` / Password: `password123` -
Username: `bob` / Password: `password123`

Testing Documentation

Running Tests

```
# Run all 74 tests
cd TaskManagementAPI.Tests
dotnet test

# Run with detailed output
dotnet test --logger "console;verbosity=detailed"

# Run specific test class
dotnet test --filter "FullyQualifiedName~AuthControllerTests"
```

Test Coverage Summary

Controller	Tests	Coverage	Description
AuthController	14	~95%	Registration, login, password hashing, JWT tokens
ProjectController	18	~95%	CRUD, search, validation, ownership, cascade delete
TaskController	25	~95%	CRUD, filtering, status transitions, assignment
CommentController	17	~95%	CRUD, ownership validation, authorization
TOTAL	74	~95%	Comprehensive API testing

Test Infrastructure

TestDbContextFactory: Creates in-memory SQLite database for each test (isolated)

```
var db = TestDbContextFactory.CreateInMemoryContext();
// Each test gets fresh database with seed data:
// - alice (ID=1), bob (ID=2), charlie (ID=3)
// - 2 projects, 2 tasks, 2 comments
```

FakeUserContext: Simulates authenticated user (avoids real JWT)

```
var userContext = new FakeUserContext(userId: 1, username: "alice");
var service = new ProjectService(db, userContext);
// Service thinks alice (ID=1) is logged in
```

Test Patterns Used

1. **Validation Tests:** Manually set `ModelState` errors to test controller validation response
2. **Authorization Tests:** Verify users can't access others' resources (returns 404)
3. **Empty List Tests:** Verify endpoints return empty arrays, not errors, when no data
4. **Cascade Delete Tests:** Verify deleting project also deletes related tasks (database integrity)
5. **Status Transition Tests:** Verify task status can change in any direction (business rules)

Full details: See `Unit_Testing_manual.md`

Security Features

Backend Security

JWT Authentication: All endpoints require valid Bearer token (like OAuth but self-hosted)

Password Hashing: BCrypt with salt (passwords never stored in plain text)

Ownership Enforcement: Service layer filters data by current user ID

Authorization Pattern: Returns 404 (not 403) for unauthorized access to avoid leaking resource existence

Input Validation: Data annotations on DTOs (string length, required fields)

SQL Injection Protection: Entity Framework parameterizes all queries

Frontend Security

Token Storage: `localStorage` (persists across sessions, cleared on logout)

Per-Request Token: Added to every HTTP call (prevents stale token issues)

No Sensitive Data in UI: User IDs and internal data not exposed in browser

Form Validation: Client-side validation before API calls (UX + security)

Response Codes

- **200 OK:** Successful GET/PUT
- **201 Created:** Successful POST
- **204 No Content:** Successful DELETE
- **400 Bad Request:** Validation failed (e.g., empty required field)
- **401 Unauthorized:** Missing or invalid JWT token

- **404 Not Found:** Resource doesn't exist OR user doesn't own it (prevents info leak)

Project Structure

TaskManagementAPI/	# Solution root
TaskManagementAPI/	# Backend API Project
Controllers/	
AuthController.cs	- Registration & login
ProjectController.cs	- Project CRUD
TaskController.cs	- Task CRUD + status/assign
CommentController.cs	- Comment CRUD
Services/	
IProjectService.cs / ProjectService.cs	
ITaskService.cs / TaskService.cs	
ICommentService.cs / CommentService.cs	
TokenService.cs	- JWT generation
IUserContext.cs / UserContext.cs	- Current user abstraction
Models/	
User.cs	- User entity
Project.cs	- Project entity
TaskItem.cs	- Task entity
Comment.cs	- Comment entity
DTOs/	
AuthDtos.cs	- Login/register DTOs
ProjectDtos.cs	- Project DTOs
TaskDtos.cs	- Task DTOs
CommentDtos.cs	- Comment DTOs
Data/	
TaskDbContext.cs	- EF Core context
Migrations/	- Database migrations
Program.cs	- App startup & DI
appsettings.json	- Configuration
TaskManagementAPI.Blazor/	# Frontend Blazor Project (Part 2)
Components/	- Razor components (see detailed tree above)
Services/	- API client services
Models/	- Frontend DTOs
wwwroot/	- Static files (CSS)
App.razor	- Root component
Program.cs	- Blazor startup
TaskManagementAPI.Tests/	# Unit Test Project

Controllers/	
AuthControllerTests.cs	- 14 tests
ProjectControllerTests.cs	- 18 tests
TaskControllerTests.cs	- 25 tests
CommentControllerTests.cs	- 17 tests
Helpers/	
FakeUserContext.cs	- Mock authenticated user
TestDbContextFactory.cs	- In-memory database setup
TaskManagementAPI.Tests.csproj	
Docs/	# Documentation
IMPLEMENTATION_PLAN_PART2.md	- Detailed dev plan
Reflection.md	- Challenges & solutions
Unit_Testing_manual.md	- Testing documentation
Assignment_Part_2_EN.txt	- Assignment requirements
TaskManagementAPI.sln	# Solution file
README.md	# This file

Lines of Code (Approximate)

- **Backend API:** ~3,500 lines (Controllers, Services, Models, DTOs)
 - **Frontend Blazor:** ~2,500 lines (Components, Services, Models)
 - **Unit Tests:** ~3,000 lines (74 tests)
 - **Total:** ~9,000 lines of C# code
-

Key Technical Decisions

Architecture Patterns

- 1. Service Layer Pattern** - Controllers are thin (routing + validation) - Services contain business logic (ownership checks, data manipulation) - **Why?** Testable, reusable, clean separation of concerns
- 2. Repository Pattern (via EF Core)** - DbContext acts as repository (no additional abstraction) - **Why?** EF Core already provides unit of work + repository patterns
- 3. DTO Pattern** - Separate DTOs for API requests/responses (not exposing entities directly) - **Why?** Prevents over-posting attacks, cleaner API contracts
- 4. Dependency Injection** - All services registered in `Program.cs` (built-in DI container) - **Why?** Loose coupling, easier testing (can swap implementations)

Frontend Patterns

1. **Component-Based Architecture** - Reusable components (like React components) - Props and events for parent-child communication - **Why?** Maintainable, testable UI
2. **Service Layer for API Calls** - Blazor components don't call HTTP directly - ApiClient → Service → Component - **Why?** Centralized error handling, token management
3. **localStorage for Token Persistence** - Not sessionStorage (disappears on tab close) - Not cookies (more complex, CSRF concerns) - **Why?** Simple, works across tabs, standard for SPAs

Database Decisions

1. **SQLite for Development** - File-based, no server setup needed - **Why?** Fast iteration, easy for team collaboration (tasks.db in repo) - **Production:** Would use PostgreSQL or SQL Server
 2. **Seed Data in Migrations** - Pre-populated users (alice, bob) for testing - **Why?** Consistent test environment, easy demo
 3. **Cascade Delete** - Deleting project deletes all tasks - **Why?** Data integrity, prevents orphaned records
-

API Testing with Swagger

Using Swagger UI

1. Run application: `dotnet run`
2. Open browser: `http://localhost:5114/swagger`
3. Click "Authorize" button (top right)
4. Get token:
 - Expand POST `/api/auth/login`
 - Click "Try it out"
 - Body: `{ "username": "alice", "password": "password123" }`
 - Execute → Copy token from response
5. Enter token in Authorize dialog: `Bearer <your-token>`
6. Now you can test any protected endpoint

Example: Create a Project

```
POST /api/project
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
{
  "name": "Mobile App Rewrite",
```

```
    "description": "Rebuild mobile app in Flutter"
  }
```

Response 201 Created:

```
{
  "id": 3,
  "name": "Mobile App Rewrite",
  "description": "Rebuild mobile app in Flutter",
  "createdAt": "2025-12-08T10:30:00Z",
  "userId": 1,
  "ownerUsername": "alice",
  "taskCount": 0
}
```

Development Notes

Database

- **SQLite** file: `TaskManagementAPI/tasks.db` (created on first run)
- Migrations in `Migrations/` folder (version-controlled schema changes)
- Seed data: 2 users (alice, bob), 2 projects, 2 tasks, 2 comments
- **Connection string:** `appsettings.json` → `ConnectionStrings:Default`

JWT Tokens

- **Algorithm:** HMAC-SHA256 (symmetric key)
- **Claims:** `sub` (user ID), `unique_name` (username)
- **Expiration:** 7 days (configurable in `TokenService`)
- **Decode online:** jwt.io (paste token to inspect claims)

Blazor Specifics

- **Render mode:** Interactive Server (prerendering disabled)
- **SignalR:** WebSocket connection for UI updates (automatic reconnection)
- **Circuit:** Each user gets a server-side “circuit” (like a session)
- **Debugging:** Browser DevTools → Console shows SignalR connection logs

CORS (Cross-Origin)

- Enabled in API for development: `AllowAnyOrigin()`
 - **Production:** Restrict to specific origins (e.g., `https://yourdomain.com`)
-

Troubleshooting

“Unauthorized” errors after login

- **Check:** JWT token stored in localStorage? (Browser DevTools → Application → Local Storage)
- **Check:** Token expired? (Tokens valid for 7 days)
- **Solution:** Logout and login again

Database not found

- **Check:** `tasks.db` file exists in `TaskManagementAPI/` folder?
- **Solution:** Run `dotnet ef database update` from API project folder

Blazor page stuck loading

- **Check:** API running on port 5114? (Blazor needs API)
- **Check:** Browser console errors? (F12 → Console)
- **Solution:** Ensure API is running, check `appsettings.json` `BaseUrl`

Tests failing

- **Check:** All 74 tests or specific ones?
 - **Solution:** Run `dotnet clean` then `dotnet build` then `dotnet test`
-

Future Improvements

Functionality

- ☐ Email verification on registration
- ☐ Password reset flow
- ☐ Task due date notifications
- ☐ File attachments on tasks
- ☐ Task dependencies (Task A blocks Task B)
- ☐ Project sharing (multiple owners)
- ☐ Activity log (audit trail)

Technical

- ☐ Paging for large lists (projects, tasks)
- ☐ Sorting (by date, name, priority)
- ☐ Real-time updates (SignalR notifications)
- ☐ Integration tests (test API + DB together)
- ☐ CI/CD pipeline (GitHub Actions)
- ☐ Docker containerization
- ☐ Rate limiting (prevent API abuse)

UI/UX

- ☐ Dark mode
 - ☐ Drag-and-drop task reordering
 - ☐ Kanban board view
 - ☐ Mobile-responsive improvements
 - ☐ Accessibility (ARIA labels, keyboard navigation)
-

Documentation References

- **Part 1 (API) Team Plan:** See git history for original team collaboration document
 - **Part 2 (Blazor) Implementation:** Docs/IMPLEMENTATION_PLAN_PART2.md
 - **Testing Documentation:** Docs/Unit_Testing_manual.md
 - **Reflection & Lessons Learned:** Docs/Reflection.md
 - **Assignment Requirements:** Docs/Assignment_Part_2_EN.txt
-

Technologies Used

Backend

- **.NET 8** - Latest LTS version of .NET
- **ASP.NET Core** - Web framework
- **Entity Framework Core** - ORM (Object-Relational Mapper)
- **SQLite** - Embedded database
- **BCrypt.Net** - Password hashing
- **System.IdentityModel.Tokens.Jwt** - JWT generation/validation

Frontend

- **Blazor Server** - UI framework
- **Blazored.LocalStorage** - Browser storage access
- **Bootstrap 5** - CSS framework
- **SignalR** - Real-time communication (built into Blazor Server)

Testing

- **xUnit** - Testing framework
- **In-memory SQLite** - Test database

Tools

- **Swagger/OpenAPI** - API documentation
- **Visual Studio 2022** - IDE

- **Git** - Version control
-

License & Academic Use

This project was developed as an educational assignment. Code may be used for learning purposes with proper attribution.

Contact

Project: Task Management System

Course: .NET Development

Date: December 2025

Repository: GitHub - TaskManagementAPI

README Version: 2.0 (Part 2 - Full-Stack Implementation)

Last Updated: December 8, 2025